



Fixed Point Solutions, LLC

Sovereign Assessment

2026/04/17

Prepared by: Kurt Barry

1. Scope

The Sovereign protocol provides automated management of DeFi positions to Safe (formerly Gnosis Safe) wallet users. Via a custom Safe module (a smart contract granted permission to execute transactions from a Safe wallet), users can configure permissions for third party actors to perform actions on their behalf. Specialized contracts referred to as connectors provide integrations with popular DeFi protocols and primitives (e.g. AAVE or generic flash loan functionality).

Repository: <https://github.com/Aphelion-Co/core-v1/>

Review commit: [d004fff2983b162084a10b085101fb74d1d0a7b2](https://github.com/Aphelion-Co/core-v1/commit/d004fff2983b162084a10b085101fb74d1d0a7b2)

Fixes reviewed through commit: [1c67cda5aa337b3a0cb20865f399287633073ea6](https://github.com/Aphelion-Co/core-v1/commit/1c67cda5aa337b3a0cb20865f399287633073ea6)

Researcher effort: 40 hours (1 researcher, 5 days) + 8 hours of fix review

Note that the Sovereign protocol has extensive integrations with external protocols. The correctness of these integrations was considered out-of-scope for this assessment. As such, most of the connector contracts were reviewed only shallowly, with most effort devoted to the core logic.

2. Limitations

No assessment can guarantee the absolute safety or security of a software-based system. Further, a system can become unsafe or insecure over time as it and/or its environment evolves. This assessment aimed to discover as many issues and make as many suggestions for improvement as possible within the specified timeframe. Undiscovered issues, even serious ones, may remain. Issues may also exist in components and dependencies not included in the assessment scope.

3. Findings

Findings and recommendations are listed in this section, grouped into broad categories. It is up to the team behind the code to ultimately decide whether the items listed here qualify as issues that need to be fixed, and whether any suggested changes are worth adopting. When a response from the team regarding a finding is available, it is provided.

Findings are given a severity rating based on their likelihood of causing harm in practice and the potential magnitude of their negative impact. Severity is only a rough guideline as to the risk an issue presents, and all issues should be carefully evaluated.

Severity Level Determination		Impact		
		High	Medium	Low
Likelihood	High	Critical	High	Medium
	Medium	High	Medium	Low
	Low	Medium	Low	Low

Issues that do not present any quantifiable risk (as is common for issues in the Code Quality category) are given a severity of **Informational**. Gas optimizations are considered a separate category and are not assigned an explicit severity.

Summary of Findings		
Severity	Count	Number Resolved
Critical	0	N/A
High	0	N/A
Medium	0	N/A
Low	3	2
Gas Optimizations	7	7
Informational	7	7

3.1 Safety and Correctness

Findings that could lead to harmful outcomes or violate the intentions of the system.

SC.1 Lack of Bitmasking in Inline Assembly of Types Less Than a Full Word

Severity: Low

Code Location:

<https://github.com/Aphelion-Co/core-v1/blob/d004fff2983b162084a10b085101fb74d1d0a7b2/src/Utils/BorrowSpreadBase.sol#L183>

<https://github.com/Aphelion-Co/core-v1/blob/d004fff2983b162084a10b085101fb74d1d0a7b2/src/Utils/BorrowSpreadBase.sol#L198>

<https://github.com/Aphelion-Co/core-v1/blob/d004fff2983b162084a10b085101fb74d1d0a7b2/src/Utils/BorrowSpreadBase.sol#L200>

Description: Solidity does not guarantee that stack words storing types of less than a full word width (256 bits) do not have dirty higher-order bits and explicitly recommends masking out the higher-order bits when accessing them in inline assembly ([reference](#)). On the lines referenced above, types of width less than 256 (variables `spread`, `to`, and `token`, respectively) are used in inline assembly without clearing their higher-order bits first.

Recommendation: Mask out the higher order bits of types of less than a full word width when using them in inline assembly as recommended by the Solidity documentation.

Response: Fixed in commit [2163938e791687389878f446eb2d6a8cecb7d6ac](#).

FPS: Fix verified.

SC.2 Rounding In Favor of the User During Borrow Spread Fee Collection Can Allow Users to Avoid Paying Some or All Borrow Spread Fees Owed

Severity: Low

Code Location:

<https://github.com/Aphelion-Co/core-v1/blob/d004fff2983b162084a10b085101fb74d1d0a7b2/src/BorrowSpreadManager.sol#L289>

<https://github.com/Aphelion-Co/core-v1/blob/d004fff2983b162084a10b085101fb74d1d0a7b2/src/BorrowSpreadManager.sol#L295>

Description: Rounding down on these two lines can result in zero fee accumulation despite updating the last index and last accrual timestamp, and always underestimates the full amount of the fee, resulting in some income loss. This allows users to do frequent, small repays or borrows to keep updating the index without paying the full borrow spread. This is easier to exploit for high-value borrow tokens or tokens with slow-accruing interest, although it will be impractical in most cases due to gas costs.

Recommendation: Consider rounding one or both of these divisions up, against the user, or employing other mitigations like not updating the index and timestamp if either calculation was zero due to truncation.

Response: Fixed in commit [bdf089bd99f78ee5b368683a790cb5f815a5b80f](#).

FPS: Fix verified.

SC.3 Connectors that Don't Exist Yet Can Be Paused

Severity: Informational

Code Location:

<https://github.com/Aphelion-Co/core-v1/blob/d004fff2983b162084a10b085101fb74d1d0a7b2/src/ConnectorRegistry.sol#L99-L107>

Description: The `ConnectorRegistry.setPaused()` function allows pausing connectors that don't exist yet (haven't been proposed and accepted), potentially creating confusion due to silently succeeding if an incorrect connector address is accidentally provided.

Recommendation: Consider checking for a non-zero `impl` and reverting the pause attempt if the check fails.

Response: Fixed in commit [8bba9ea7a34174cf419db55270795f5da4deae0a](#).

FPS: Fix verified.

SC.4 Missing Event Emission for Possible Failed Call to BorrowSpreadManager

Severity: Informational

Code Location:

<https://github.com/Aphelion-Co/core-v1/blob/d004fff2983b162084a10b085101fb74d1d0a7b2/src/Utils/BorrowSpreadBase.sol#L108-L113>

Description: In all other instances where a call to the `BorrowSpreadManager` could fail in `BorrowSpreadBase`, a `BorrowSpreadCallFailed` event is emitted upon a failure, but not here.

Recommendation: Emit a `BorrowSpreadCallFailed` event if this call fails as well.

Response: Fixed in commit [76251a3204fc60efa43384b86cdddda3d5ff0c40](#).

FPS: Fix verified.

SC.5 Misleading and Suboptimal Errors

Severity: Informational

Code Location:

[1]<https://github.com/Aphelion-Co/core-v1/blob/d004fff2983b162084a10b085101fb74d1d0a7b2/src/SafeFactory.sol#L153>

[2]<https://github.com/Aphelion-Co/core-v1/blob/d004fff2983b162084a10b085101fb74d1d0a7b2/src/SafeFactory.sol#L163>

[3]<https://github.com/Aphelion-Co/core-v1/blob/d004fff2983b162084a10b085101fb74d1d0a7b2/src/SafeFactory.sol#L166>

[4]<https://github.com/Aphelion-Co/core-v1/blob/d004fff2983b162084a10b085101fb74d1d0a7b2/src/SafeModuleAuth.sol#L263>

[5]<https://github.com/Aphelion-Co/core-v1/blob/d004fff2983b162084a10b085101fb74d1d0a7b2/src/SafeModuleAuth.sol#L267>

Description and Recommendation:

[1,2,3] `BadAddress()` is not really an appropriate error for these three exceptional conditions. Consider more descriptive errors for easier debugging.

[4,5] `WrongRecipe()` doesn't seem like an appropriate error for these two exceptional conditions. Consider more descriptive errors for easier debugging.

Response: Fixed in commit [f1523438573171bfa6f038d2b12be35dd01b0766](https://github.com/Aphelion-Co/core-v1/commit/f1523438573171bfa6f038d2b12be35dd01b0766).

FPS: Fix verified.

3.2 Usability and Incentives

Findings that could lead to suboptimal user experience, hinder integrations, or lead to undesirable behavioral outcomes.

U.1 Assessing the Borrow Spread Fee In Proportion to the Interest Paid Potentially Discourages Use

Severity: Low

Description: The borrow spread fee is assessed as a percentage of the interest charged. For example, if the borrow spread fee is 10%, a user paying 1% annual interest on their borrow effectively pays 1.1%; a user paying 10% effectively pays 11%. The user with the more costly borrow is paying significantly more, for no clearly justifiable reason. The Sovereign protocol does not take on any risk of the underlying position, and automation bots are already compensated in other ways for position management (and the higher borrow rate position could potentially have a larger safety margin, and hence require less management than the lower borrow rate position). Further, if both positions are part of interest rate arbitrage trades, the

threshold for rate differential that allows profitability is much higher for tokens with steeper borrow rates. All told, this mechanism design strongly encourages borrowers seeking to borrow tokens with high interest rates to perform some or all of their position operations outside of the Sovereign protocol to route around the disproportionate fee imposed upon them.

Recommendation: Consider alternative fee structures. The simplest would be a constant percent, e.g. and additional 0.1% so that the two users in the example above pay 1.1% and 10.1%, respectively. This could be undesirable from another perspective: for very low interest rates, a constant borrow spread fee could again be disproportionate, especially if a very small rate differential is being arbitrated. To address this, the next simplest option would be to scale the borrow spread fee linearly up to some cap (any function with a horizontal asymptote could also be used, e.g. inverse tangent, but from a complexity standpoint a piecewise linear function is likely preferable even if not theoretically optimal).

Response: Acknowledged.

3.3 Gas Optimizations

Findings that could reduce the gas costs of interacting with the protocol, potentially on an amortized or averaged basis. These findings are generally not given a severity; those that reduce storage writes, reads, and external calls are generally the highest impact. Complex operations, especially involving memory manipulation, that are done in loops can be significant as well.

G.1 Reorder Fields in `ConnectorRegistry.PendingUpdate` to Use One Fewer Storage Slot

Code Location:

<https://github.com/Aphelion-Co/core-v1/blob/d004fff2983b162084a10b085101fb74d1d0a7b2/src/ConnectorRegistry.sol#L17-L21>

Description and Recommendation: Re-ordering the fields of the `PendingUpdate` struct so that `exists` and `next` are adjacent will allow them to pack into a single storage slot, reducing the slots needed for this struct from three to two. Or, using a smaller numerical type for `validAfter` would allow only a single slot to be used.

Response: Fixed in commit [b53e249cfcafb04d37127b93cda04a3813586aae](#).

FPS: Fix verified (only one slot used; the unchecked cast in `propose()` should not be a concern since `delay` is `immutable` and can easily be chosen to be small enough to bound `block.timestamp + delay <= type(uint64).max` for any reasonable value of `block.timestamp`).

G.2 Reorder Fields in `BorrowSpreadManager.Position` to Use One Fewer Storage Slot

Code Location:

<https://github.com/Aphelion-Co/core-v1/blob/d004fff2983b162084a10b085101fb74d1d0a7b2/src/BorrowSpreadManager.sol#L28-L35>

Description and Recommendation: Re-ordering the fields in the `Position` struct so that `token` and `lastAccrualTs` are adjacent will allow them to pack into a single storage slot, reducing the slots needed for this struct by one.

Response: Fixed in commit [b999708e9700b198c233f19370f891d4d27bd643](https://github.com/Aphelion-Co/core-v1/commit/b999708e9700b198c233f19370f891d4d27bd643).

FPS: Fix verified.

G.3 `ConnectorRegistry.rollback()` Re-fetches the Codehash of the Previous Implementation Unnecessarily

Code Location:

<https://github.com/Aphelion-Co/core-v1/blob/d004fff2983b162084a10b085101fb74d1d0a7b2/src/ConnectorRegistry.sol#L93>

Description and Recommendation: When rolling back a connector implementation, `_pinCodehash(id, prev)` is invoked to reset the codehash to that of the previous implementation. This function is a thin wrapper around an `extcodehash` call followed by a storage write. The `rollback()` function previously fetched the old implementation's codehash on line 88 and stored it in the `actualHash` variable; thus this line could be replaced with `_codehash[id] = actualHash;`, saving another `extcodehash` operation.

Response: Fixed in commit [efa2548011a9c5d2d2a0e6cbe884fcfa1ff14baf](https://github.com/Aphelion-Co/core-v1/commit/efa2548011a9c5d2d2a0e6cbe884fcfa1ff14baf).

FPS: Fix verified.

G.4 Checks Make `safe` Argument to `Executor.executeRecipe()` Redundant

Code Location:

<https://github.com/Aphelion-Co/core-v1/blob/d004fff2983b162084a10b085101fb74d1d0a7b2/src/Executor.sol#L121-L122>

Description and Recommendation: These two checks enforce that the `safe` value used is always equal to `msg.sender`. The `safe` argument could be eliminated and `msg.sender` used directly, reducing gas costs.

Response: Fixed in commit [cfcc9316a596c568349d7b1e546a5fae6f20b437](https://github.com/Aphelion-Co/core-v1/commit/cfcc9316a596c568349d7b1e546a5fae6f20b437).

FPS: Fix verified.

G.5 Unnecessary Check in `Executor._runPostchecks()`

Code Location:

<https://github.com/Aphelion-Co/core-v1/blob/d004fff2983b162084a10b085101fb74d1d0a7b2/src/Executor.sol#L402>

Description and Recommendation: All code paths leading to this check result in `pre` and `postchecks` having the same length by construction. The check could be eliminated.

Response: Fixed in commit [84b6a2eef32e186069092427f9f77097fea6830](https://github.com/Aphelion-Co/core-v1/commit/84b6a2eef32e186069092427f9f77097fea6830).

FPS: Fix verified.

G.6 `AutomationStore.setEnabledByExecutor()` Can Be Simplified

Code Location:

<https://github.com/Aphelion-Co/core-v1/blob/d004fff2983b162084a10b085101fb74d1d0a7b2/src/AutomationStore.sol#L239>

Description and Recommendation: This function is only used for disabling, never enabling. Assuming there is no possibility this will change, the boolean argument could be eliminated and the behavior simplified.

Response: Fixed in commit [ccb5ca592e308faa1cfe72cdca761172598a9690](https://github.com/Aphelion-Co/core-v1/commit/ccb5ca592e308faa1cfe72cdca761172598a9690).

FPS: Fix verified.

G.7 Inefficient and Unnecessary Function to Compute Powers of 10 With Overflow Checking

Code Location:

<https://github.com/Aphelion-Co/core-v1/blob/d004fff2983b162084a10b085101fb74d1d0a7b2/src/Utils/Decimals.sol#L12-L22>

Description and Recommendation: The built-in `**` operator can be used instead of this function and should use a more efficient algorithm than iterative multiplication (e.g. repeated squaring), and is checked for overflow by default. If there's a strong desire to emit a custom error for overflow, the computation of the result should still be simplified to use `**` wrapped in an unchecked block.

Response: Fixed in commit [4084d9f412b5f42736afc995e867aebd83684dbb](#).

FPS: Fix verified.

3.4 Code Quality

CQ.1 Unused Imports

Severity: Informational

Code Location:

[1]<https://github.com/Aphelion-Co/core-v1/blob/d004fff2983b162084a10b085101fb74d1d0a7b2/src/GasOracleRegistry.sol#L6>

Description:

[1] `BadFeed`, `StalePrice`

Recommendation: Remove unused imports (or use them appropriately if there is missing logic).

Response: Fixed in commit [3fe02c22c54164ad8a7325f4bad67b4904ae94df](#).

FPS: Fix verified.

CQ.2 Documentation Issues

Severity: Informational

Code Location:

[1]<https://github.com/Aphelion-Co/core-v1/blob/d004fff2983b162084a10b085101fb74d1d0a7b2/src/SafeModuleAuth.sol#L55-L56>

[2]<https://github.com/Aphelion-Co/core-v1/blob/d004fff2983b162084a10b085101fb74d1d0a7b2/src/connectors/SwapConnector.sol#L43-L44>

Description and Recommendation:

[1] There are comments on the lines above describing the purpose of all the other transient fields; add comments for these two fields as well for completeness.

[2] Missing documentation for the `EXACT_INPUT` action type.

Response: Fixed in commit [d4398e0362fae317f80aefeadce1714a99b52240](#) and commit [599407adf98c3798db64c0020a2c2d1bd6b03659](#).

FPS: Fix verified.

CQ.3 Minor Redundancies

Severity: Informational

Code Location:

[1]<https://github.com/Aphelion-Co/core-v1/blob/d004fff2983b162084a10b085101fb74d1d0a7b2/src/FeeManager.sol#L164-L170>

Description and Recommendation:

[1] These two functions differ only in name; one should be removed to reduce code size.

Response: Fixed in commit [884af5ca68edc177fb3c47ce1803050f0cdd1a82](#).

FPS: Fix verified.

CQ.4 Readability Enhancements

Severity: Informational

Code Location:

[1]<https://github.com/Aphelion-Co/core-v1/blob/d004fff2983b162084a10b085101fb74d1d0a7b2/src/BorrowSpreadManager.sol#L202-L212>

Description and Recommendation:

[1] Group this function with the other `view` functions.

Response: Fixed in commit [cd973850577e50457a3e50dac088aeea45766d8a](#).

FPS: Fix verified.

4. Notes

This section contains general considerations for interacting with or maintaining the system and various conclusions reached or discoveries made during the course of the assessment. Whereas findings generally represent things for the team to consider changing, notes are more informational and may be helpful to those who intend to interact with the system.

4.1 Failing Tests

At the frozen review commit, two tests were discovered to be failing during routine test running. The project team investigated, and provided fixes for both. One involved only test changes, but the second required a contract code change to support arbitrary oracle heartbeat frequencies. Both commits were reviewed in addition to the fixes made to the issues identified above.

The `test_forkbug_connector_drains_aave_position` test was fixed in commit [ce95084d562c12791ef633778aef489f5f817ae8](#) and involved only test changes.

The `test_0x_swap_weth_to_usdc` test was fixed in commit [f29a6bb69303013d23e44aae72db9fc705bce6dd](#) and required the addition of an oracle maximum age parameter to the `AggregatorSwap` connector.

The team was initially unaware of these failures because they were not being run in CI; the fix commits also ensure the CI will run these tests.